Correção · Prova de Engenharia de Software · 15 questões

Data: 26/10/2026 · **Semana 1** · **Placar: 8/15 (53%)** · 6 erros · 1 em branco **Respostas lidas das anotações do Okular:** 1-E · 2-B · 3-A · 4-B · 5-(branco) · 6-E · 7-B · 8-A · 9-A · 10-B · 11-B · 12-A · 13-D · 14-D · 15-E

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais. Base: Sommerville, Pressman, Guia do Scrum 2020.

Questão	Assunto	Sua	Gabarito	Resultado
1	Espiral × prototipação	E	C	❌
2	Modelo em V	B	A	❌
3	Prototipação ("incorreta")	A	D	❌
4	Fases do RUP	B	B	✅
5	Perspectivas do RUP	—	D	⬜ em branco
6	RUP × ágil	E	E	✅
7	Verificação × validação	B	B	✅
8	Scrum: quem decide o quê	A	C	❌
9	Artefatos e compromissos	A	A	✅ (chute, segundo sua anotação)
10	Práticas da XP	B	B	✅
11	Scaling up × out	B	E	❌
12	Subtipos de RNF	A	A	✅
13	Kanban: lead × cycle time	D	C	❌
14	Leitura de código: TDD (Python)	D	D	✅
15	Leitura de código: refatoração (Java)	E	E	✅

Por bloco: Modelos de processo 0/3 · RUP 2/3 (1 em branco) · V&V 1/1 · Scrum 1/2 · XP/escala 1/2 · RNF 1/1 · Kanban 0/1 · Código 2/2

Comparação com o resultado anterior (Eng. Software, 24/10): 9/14 (64%) → 8/15 (53%). A prova de hoje foi mais difícil: 8 das 15 eram sobre pontos fracos. Houve **evolução** em RUP fases (branco → acerto), XP práticas, subtipos de RNF e leitura de código (2/2). Houve **repetição de erro** em perspectivas do RUP (branco de novo), scaling up × out e responsabilidades no Scrum.

Questão 1 · ❌ Errou

Espiral × prototipação

Um Tribunal de Justiça decidiu migrar seu sistema processual legado para uma nova plataforma, com uma tecnologia que a equipe nunca utilizou. O projeto é caro e cercado de incertezas técnicas. A direção de TI exige que, a cada ciclo, sejam redefinidos os objetivos, identificados e reduzidos os principais riscos, inclusive com a construção de protótipos, e que ao fim de cada ciclo se decida se o projeto deve continuar.

O modelo de processo mais adequado a essa exigência é o

(A) cascata, pois as fases sequenciais com aprovação formal eliminam a incerteza tecnológica antes da codificação.

(B) modelo em V, pois associa cada fase de especificação a um nível de teste correspondente.

✅ **(C) espiral, pois cada volta inclui a análise e a redução de riscos e permite replanejar ou encerrar o projeto.**

(D) incremental, pois a primeira entrega contém o núcleo do produto, o que elimina os riscos técnicos do projeto.

❌ (E) de prototipação descartável, pois o protótipo construído evolui até se tornar o sistema final. ← sua resposta

Onde você se confundiu: você sublinhou "construção de protótipos" e riscou a (C). A palavra "protótipo" puxou para a (E), mas no espiral o protótipo é **ferramenta para reduzir risco** dentro de cada volta. As pistas decisivas eram "**riscos**", "**a cada ciclo**" e "**decidir se o projeto deve continuar**". Além disso, a (E) se contradiz: protótipo **descartável** não evolui até virar o sistema. Quem evolui é o protótipo **evolutivo**.

📌 **Espiral = risco + voltas + decidir se continua.** Usar protótipo dentro das voltas não transforma o modelo em "prototipação". E "descartável que evolui" é contradição.

Questão 2 · ❌ Errou

Modelo em V

Uma equipe adotou o modelo em V para desenvolver o sistema de controle de depósitos judiciais. Sobre esse modelo, analise as afirmativas a seguir.

I. O teste de aceitação é planejado a partir dos requisitos do usuário, que estão no topo do lado esquerdo do V. → **Verdadeira**

II. O teste unitário corresponde à especificação de requisitos do sistema, pois verifica se cada requisito foi atendido. → **Falsa**

III. Por associar um nível de teste a cada fase, o modelo em V é iterativo, permitindo revisitar as fases a cada ciclo de entrega. → **Falsa**

Está correto o que se afirma em

✅ **(A) I, apenas.**

❌ (B) II, apenas. ← sua resposta

(C) I e II, apenas.

(D) I e III, apenas.

(E) I, II e III.

Onde você se confundiu: você acertou o item III (anotou "sequencial") 👏. No item I, você anotou "lado direito". O **teste** de aceitação fica mesmo à direita, mas a frase diz que os **requisitos do usuário** estão no topo do lado esquerdo, e isso está certo. Foi uma leitura de "a quem se refere" a expressão. No item II, o teste unitário corresponde ao **projeto detalhado / módulo**. Quem corresponde aos requisitos do sistema é o **teste de sistema**.

📌 **V = esquerda desce (especificação), direita sobe (testes), na mesma altura:** requisitos do usuário ↔ **aceitação** · requisitos do sistema/arquitetura ↔ **sistema** · projeto de módulos ↔ **integração** · código ↔ **unitário**. O V é **sequencial**.

Questão 3 · ❌ Errou

Prototipação · "assinale a incorreta"

A equipe de sistemas de um tribunal pretende usar prototipação no desenvolvimento do novo portal de peticionamento eletrônico.

Sobre a prototipação, assinale a afirmativa **incorreta**.

❌ (A) O protótipo apoia a elicitação e a validação de requisitos, pois os usuários reagem melhor a algo concreto do que a uma descrição em papel. ← sua resposta

(B) O protótipo descartável é construído para esclarecer requisitos e, depois de cumprir esse papel, é abandonado.

(C) Um risco da prototipação é o cliente acreditar que o protótipo já é praticamente o sistema final.

✅ **(D) O protótipo descartável deve ser aproveitado como base do sistema final, pois já incorpora os requisitos não funcionais de desempenho e segurança validados com o usuário.**

(E) Na prototipação evolutiva, o protótipo é refinado sucessivamente até se tornar o sistema entregue.

Onde você se confundiu: a estratégia V/F funcionou: você deu ✓ em B, C e E e pôs "?" na (D), que era a resposta. Você destacou "elicitação e validação" na (A), provavelmente achando que protótipo serve só para validar. Para Sommerville, o protótipo ajuda **nas duas coisas**: na elicitação (o usuário descobre o que quer ao ver a tela) e na validação. A (D) tem dois erros: o descartável **não** vira base do sistema, e ele normalmente **ignora** os RNF (desempenho e segurança).

📌 **Prototipação ajuda na elicitação E na validação.** Protótipo descartável **não deve** virar o sistema final, porque ignora RNF e tem estrutura ruim. Na dúvida entre um "?" e uma alternativa que você achou parcialmente estranha, fique com a que tem **erro conceitual claro** (a D tinha dois).

Questão 4 · ✅ Acertou

Fases do RUP

Em um projeto conduzido segundo o RUP para o novo sistema de execução fiscal, ao final de uma das fases a equipe apresentou:

- a descrição da arquitetura, validada por um protótipo arquitetural executável;
- cerca de 80% dos casos de uso descritos;
- os riscos principais do projeto tratados;
- o plano de desenvolvimento para a etapa seguinte.

A fase concluída foi a de

(A) Concepção.

✅ **(B) Elaboração.** ← sua resposta

(C) Construção.

(D) Transição.

(E) Implantação, quinta fase do RUP, dedicada à validação da arquitetura.

Em 24/10 você deixou essa em branco. Hoje acertou e riscou as outras corretamente. A Concepção entrega o **caso de negócio**, a Construção entrega o **software operacional** e a Transição **implanta**. A (E) não existe: o RUP tem 4 fases, e "implantação" é uma **disciplina**.

📌 **Elaboração = arquitetura + maioria dos casos de uso + riscos principais tratados + plano.**

Questão 5 · ⬜ Em branco ("N sei")

Perspectivas do RUP

Sommerville descreve o RUP a partir de três perspectivas. Analise as afirmativas a seguir.

I. A perspectiva dinâmica mostra as fases do modelo ao longo do tempo. → **Verdadeira**

II. A perspectiva estática mostra as atividades (workflows ou disciplinas) realizadas no processo, que não estão associadas a uma fase específica. → **Verdadeira**

III. A perspectiva prática estabelece que cada disciplina ocorre em uma única fase, como a de requisitos, que se limita à Concepção. → **Falsa**

Está correto o que se afirma em

(A) I, apenas.

(B) II, apenas.

(C) III, apenas.

✅ **(D) I e II, apenas.**

(E) I, II e III.

⬜ Sua resposta: em branco ("? N sei 😞")

O que faltou: é a 2ª vez em branco nesse assunto (24/10 e hoje). A regra já está no `regras-tjpb.md` desde 24/10, mas ainda não foi fixada. Mesmo sem saber, dava para **eliminar o III**: você já sabe que o RUP é iterativo, então uma disciplina "limitada a uma única fase" contradiz isso. Eliminando o III, ficam A, B e D. Como I e II descrevem coisas diferentes, que podem ser verdadeiras ao mesmo tempo, a (D) era o melhor chute.

📌 **RUP · 3 perspectivas: Dinâmica = QUANDO (fases) · Estática = O QUÊ (disciplinas/workflows) · Prática = COMO FAZER BEM (6 boas práticas).** Toda disciplina pode ocorrer em qualquer fase. Na prova FGV, **nunca deixe em branco**: não há desconto por erro.

Questão 6 · ✅ Acertou

RUP × ágil

Durante uma reunião, um gestor afirmou que o tribunal "já é ágil, porque usa o RUP há anos". Sobre o RUP e sua relação com os métodos ágeis, assinale a afirmativa correta.

(A) O RUP é um método ágil, pois é iterativo e incremental, características que definem a agilidade.

(B) O RUP é um processo sequencial, em que a fase de Construção só se inicia após o encerramento de todas as atividades de requisitos.

(C) Por ser posterior ao Manifesto Ágil, o RUP incorporou os quatro valores do manifesto à sua definição.

(D) AUP e OpenUP são apenas nomes alternativos do RUP original, sem alteração no processo.

✅ **(E) O RUP é dirigido a casos de uso, centrado na arquitetura, iterativo e incremental, mas é classificado como processo dirigido a planos.** ← sua resposta

Você riscou certo a (A) (iterativo não é igual a ágil), a (B) (o RUP não é sequencial) e a (D) (AUP e OpenUP são variações enxutas). A (C) erra a data: o RUP é do fim dos anos 1990, **antes** do Manifesto (2001).

📌 **RUP: iterativo e incremental, mas dirigido a planos. Não é ágil.**

Questão 7 · ✅ Acertou

Verificação × validação

Na homologação do sistema de cumprimento de mandados, a equipe confirmou que todas as funções implementadas correspondem exatamente ao documento de requisitos aprovado. Mesmo assim, os

oficiais de justiça relataram que o sistema não serve para o trabalho de campo, pois exige conexão permanente com a internet, inexistente em várias zonas rurais.

Considerando a distinção clássica de Boehm entre verificação e validação, é correto afirmar que

(A) tanto a verificação quanto a validação falharam, pois o sistema não atende aos oficiais de justiça.

✅ **(B) a verificação foi bem-sucedida, pois o produto foi construído corretamente segundo a especificação, mas a validação revelou que não se construiu o produto certo para a necessidade real dos usuários.** ← sua resposta

(C) a validação foi bem-sucedida, pois o sistema foi homologado, mas a verificação falhou, pois os usuários não ficaram satisfeitos.

(D) a validação confere o sistema contra a especificação e a verificação o confere contra a necessidade real; por isso, a verificação falhou.

(E) como os requisitos foram aprovados formalmente, não há falha de verificação nem de validação, mas apenas uma demanda de evolução.

Ótimo. Você sublinhou exatamente as pistas: "correspondem ao documento de requisitos" (verificação ok) e "não serve para o trabalho de campo" (validação falhou). A (D) inverte os conceitos, e a (A) e a (E) ignoram a diferença entre eles.

📌 **Verificação = produto feito corretamente (contra a especificação) · Validação = produto certo (contra a necessidade real).**

Questão 8 · ❌ Errou

Scrum: quem decide o quê

Em um Scrum Team de um tribunal, observam-se as seguintes situações:

I. O Product Owner reordena os itens do Product Backlog após conversar com a Corregedoria sobre as prioridades do semestre. → **Verdadeira**

II. O Scrum Master define quais Developers executarão cada item do Sprint Backlog, para acelerar a entrega. → **Falsa**

III. Durante a Sprint, os Developers ajustam o plano do Sprint Backlog à medida que aprendem mais sobre o trabalho, sem alterar a Sprint Goal. → **Verdadeira**

Está de acordo com o Guia do Scrum 2020 o que se descreve em

❌ (A) I, apenas. ← sua resposta

(B) II, apenas.

✅ **(C) I e III, apenas.**

(D) II e III, apenas.

(E) I, II e III.

Onde você se confundiu: no item III você anotou "quem altera é o PO". O PO é dono do **Product** Backlog. O **Sprint** Backlog é dos **Developers**: o Guia 2020 diz que é "um plano feito por e para os Developers" e que ele é atualizado ao longo da Sprint conforme se aprende mais. O que não muda é a **Sprint Goal**. Se o escopo precisar mudar, os Developers **negociam** com o PO, mas o plano é deles. O item II você eliminou certo: o SM não distribui tarefas.

📌 **Product Backlog → PO ordena · Sprint Backlog → Developers ajustam durante a Sprint (a Sprint Goal fica fixa) · SM não distribui tarefas.**

Questão 9 · ✅ Acertou (segundo sua anotação, foi chute)

Artefatos e compromissos

Sobre os artefatos do Scrum e seus compromissos, segundo o Guia do Scrum 2020, assinale a afirmativa correta.

✅ **(A) Cada artefato contém um compromisso: o Product Backlog, a Meta do Produto (Product Goal); o Sprint Backlog, a Meta da Sprint (Sprint Goal); e o Incremento, a Definição de Pronto (DoD).** ← sua resposta

(B) A Definição de Pronto (DoD) é o quarto artefato do Scrum, ao lado do Product Backlog, do Sprint Backlog e do Incremento.

(C) A Meta da Sprint é o compromisso do Product Backlog, e a Meta do Produto é o compromisso do Sprint Backlog.

(D) O refinamento do Product Backlog é um evento formal do Scrum, com timebox de 4 horas para uma Sprint de um mês.

(E) Um Incremento só pode ser entregue às partes interessadas durante a Sprint Review.

Você anotou "lembrar de estudar mais essa parte, foi praticamente um chute". Por isso o acerto **não conta** para tirar o assunto dos pontos fracos. A (B) é a pegadinha clássica (DoD é compromisso, não artefato). A (C) inverte os pares. A (D) erra porque refinamento não é evento. A (E) erra porque um Incremento pode ser entregue **antes** da Review.

📌 **3 artefatos → 3 compromissos: PB → Product Goal · SB → Sprint Goal · Incremento → DoD.** Truque: o **G**oal maior vai com o backlog maior; o "pronto" vai com o que fica **pronto** (Incremento).

Questão 10 · ✅ Acertou

Práticas da XP

Uma equipe de desenvolvimento afirma seguir a Programação Extrema (XP). Observe as práticas descritas pela equipe:

I. Cada módulo tem um desenvolvedor responsável, o único autorizado a alterar o seu código. → **Incompatível**

II. Nas semanas de entrega, horas extras por várias semanas seguidas são a regra, para não atrasar o release. → **Incompatível**

III. Um servidor da vara fica disponível junto da equipe para esclarecer dúvidas e escrever as histórias de usuário. → **Compatível**

IV. O código é integrado várias vezes ao dia, e o build só é aceito se todos os testes automatizados passarem. → **Compatível**

São compatíveis com as práticas da XP apenas

(A) I e II.

✅ **(B) III e IV.** ← sua resposta

(C) I, III e IV.

(D) II, III e IV.

(E) I, II, III e IV.

Em 24/10 você aceitou "propriedade coletiva = um dono" e errou. Hoje anotou "acho que não, programação coletiva" e riscou o item I 👏. O II viola o **ritmo sustentável**, o III é o **cliente no local** e o IV é a **integração contínua**.

📌 **XP: propriedade coletiva (sem dono de módulo) · ritmo sustentável (sem horas extras seguidas) · cliente no local · integração contínua.**

Questão 11 · ❌ Errou

Scaling up × scaling out

Após o sucesso de uma equipe Scrum no módulo de audiências, a direção de TI do tribunal decidiu:

1. usar métodos ágeis no desenvolvimento de um novo sistema de grande porte, que envolverá oito equipes;
2. levar a abordagem ágil a todas as diretorias de TI do tribunal, que têm décadas de cultura de desenvolvimento dirigido a planos.

Segundo Sommerville, as decisões 1 e 2 correspondem, respectivamente, a

(A) scaling out e scaling up.

❌ (B) scaling up e scaling up. ← sua resposta

(C) scaling out e scaling out.

(D) SAFe e Scrum of Scrums, que são os únicos nomes que Sommerville usa para o escalamento ágil.

✅ **(E) scaling up e scaling out.**

Onde você se confundiu: é o 2º erro nesse assunto (24/10 e hoje). Você acertou o item 1 (**up**: sistema grande, várias equipes). No item 2, você sublinhou "todas as diretorias" e "décadas", que são justamente as pistas de **scaling out**: espalhar o ágil pela **organização**, enfrentando a cultura existente. Quando a questão usa um mesmo termo para duas situações diferentes, desconfie: a FGV raramente faz isso.

📌 **Scaling UP = o SISTEMA cresce (projeto grande, várias equipes) · Scaling OUT = o ágil se espalha para FORA, pela ORGANIZAÇÃO inteira (cultura, outras áreas).** Imagem: up = prédio mais alto; out = mais prédios no bairro.

Questão 12 · ✅ Acertou

Subtipos de RNF

No levantamento de requisitos do novo sistema de consulta processual, foram registrados os seguintes requisitos não funcionais:

1. "O sistema deve responder às consultas em até 2 segundos para 95% das requisições."
2. "O sistema deve ser desenvolvido em Java, conforme norma interna da Secretaria de Tecnologia da Informação."
3. "O tratamento dos dados pessoais das partes deve observar a Lei Geral de Proteção de Dados (LGPD)."

Segundo a classificação de Sommerville, os requisitos 1, 2 e 3 são, respectivamente, de

✅ **(A) produto, organizacional e externo.** ← sua resposta

(B) produto, externo e organizacional.

(C) organizacional, produto e externo.

(D) externo, organizacional e produto.

(E) produto, organizacional e organizacional, pois a LGPD é cumprida por exigência do próprio tribunal.

Em 24/10 você inverteu os subtipos. Hoje acertou e eliminou certo a (E): a LGPD vem **de fora** (lei), mesmo que o tribunal a cumpra.

📌 **"Quem exige isso?" → o software (produto) · a organização (organizacional) · a lei ou o regulador (externo).**

Questão 13 · ❌ Errou

Kanban: lead time × cycle time

A equipe de sustentação do PJe usa um quadro Kanban com as colunas "A fazer", "Fazendo", "Em teste" e "Concluído". Um chamado foi registrado e entrou na coluna "A fazer" na segunda-feira às 9h. Ficou parado até quarta-feira às 9h, quando um desenvolvedor o puxou para "Fazendo". O chamado chegou a "Concluído" na sexta-feira às 9h.

Considerando dias corridos, o lead time e o cycle time desse chamado foram, respectivamente,

(A) 2 dias e 4 dias.

(B) 4 dias e 4 dias.

✅ **(C) 4 dias e 2 dias.**

❌ (D) 2 dias e 2 dias. ← sua resposta

(E) 6 dias e 2 dias.

Onde você se confundiu: você marcou "2" entre segunda e quarta e tomou esse intervalo como lead time. Esse intervalo é só o **tempo de espera na fila**. O lead time vai da **entrada do pedido** (seg 9h) até a **entrega** (sex 9h), ou seja, **4 dias**. O cycle time vai do **início do trabalho** (qua 9h) até a entrega, ou seja, **2 dias**. Kanban ainda não tinha caído em nenhuma questão, então é um assunto novo para a lista de pontos fracos.

📌 **Lead time = pedido → entrega (visão do cliente, inclui a fila) · Cycle time = início do trabalho → entrega (visão do time).** Lead ≥ cycle, sempre. Se a alternativa der lead **menor** que cycle, está errada.

Questão 14 · ✅ Acertou · Leitura de código

TDD em Python

Uma equipe que pratica desenvolvimento orientado a testes (test-first) implementa o cálculo de custas processuais em Python. A regra é: custa mínima de R$ 50 para causas de até R$ 1.000; acima disso, 5% do valor da causa, **limitado a um teto de R$ 3.000**. A equipe acabou de escrever o teste `test_teto` e ainda não alterou a função.

```
def custas(valor_causa):
    if valor_causa <= 1000:
        return 50
    return valor_causa * 0.05

def test_minimo():
    assert custas(1000) == 50

def test_percentual():
    assert custas(2000) == 100

def test_teto():
    assert custas(100000) == 3000
```

Ao executar os três testes, é correto afirmar que

(A) os três testes passam, e a equipe está na etapa verde, devendo agora refatorar o código.

(B) `test_percentual` falha, pois `2000 * 0.05` retorna o valor `100.0`, que o Python considera diferente de `100`; a equipe está na etapa vermelha.

(C) `test_minimo` e `test_teto` falham, pois a condição `<= 1000` exclui o valor 1000 da faixa mínima.

✅ **(D) apenas `test_teto` falha, pois a função retorna `5000.0`; a equipe está na etapa vermelha, e o próximo passo é escrever o código mínimo que faça o teste passar.** ← sua resposta

(E) apenas `test_teto` falha, e o próximo passo correto do ciclo é refatorar o código existente antes de fazer o teste passar.

Excelente rastreio: você sublinhou o `5000.0` e o teto. O cálculo é: `custas(1000)` → 50 ✓ · `custas(2000)` → `100.0`, e em Python `100.0 == 100` é **True** ✓ (a B é pegadinha) · `custas(100000)` → `5000.0` ≠ 3000 ✗. A (C) erra porque `<=` **inclui** o 1000. A (E) inverte a ordem do ciclo.

📌 **TDD: vermelho (teste novo falha) → verde (código mínimo) → refatorar.** Não se refatora com teste falhando. Em Python, `100.0 == 100` é `True`.

Questão 15 · ✅ Acertou · Leitura de código

Refatoração em Java

Considere o método Java a seguir, que soma as custas válidas de um processo:

```
double total(List<Double> custas) {
    double soma = 0;
    for (double c : custas) {
        if (c > 0) {
            soma += c;
        }
    }
    return soma;
}
```

Sabendo que refatorar é alterar a estrutura interna do código sem alterar o seu comportamento observável, a alteração que **não** constitui uma refatoração é

(A) renomear a variável `soma` para `totalCustas` em todas as suas ocorrências.

(B) mover o método para uma nova classe `CalculadoraCustas`, mantendo a assinatura e atualizando todas as chamadas.

(C) extrair a condição para um método `boolean ehValida(double c) { return c > 0; }` e usá-lo no `if`.

(D) substituir o for-each por `for (int i = 0; i < custas.size(); i++)`, acessando cada elemento com `custas.get(i)`.

✅ **(E) remover o `if` e somar todos os elementos da lista, já que, em tese, não deveriam existir custas negativas.** ← sua resposta

Perfeito: você circulou o `if` e sublinhou "custas negativas". Com a lista `[100, -20]`, o original retorna 100 e a versão (E) retorna 80: o **comportamento muda**. A, B, C e D são refatorações clássicas: renomear, mover método, extrair método e trocar a forma do laço.

📌 **Refatoração não muda a saída para nenhuma entrada.** Teste rápido: invente uma entrada "estranha" (negativo, vazio, nulo) e compare as duas versões.

Para o caderno de erros

Assunto	Por que errei	Regra certa
Espiral × prototipação (Q1)	A palavra "protótipo" me puxou para prototipação	Espiral = risco + voltas + decidir se continua. Protótipo é ferramenta dentro da volta
Modelo em V (Q2)	Confundi o lado do requisito com o lado do teste; liguei unitário a requisitos do sistema	Aceitação ↔ req. usuário · sistema ↔ req. sistema · integração ↔ projeto · unitário ↔ código
Prototipação (Q3)	Achei que protótipo serve só para validar	Protótipo ajuda na elicitação E na validação. Descartável não vira sistema
Perspectivas do RUP (Q5)	Não fixei e deixei em branco (2ª vez)	Dinâmica = quando · estática = o quê · prática = como fazer bem. Nunca deixar em branco
Scrum (Q8)	Achei que o PO altera o Sprint Backlog	Sprint Backlog é dos Developers; a Sprint Goal fica fixa
Scaling (Q11)	Usei "up" para a organização inteira (2ª vez)	Up = sistema grande · Out = organização inteira
Kanban (Q13)	Tomei o tempo de fila como lead time	Lead = pedido → entrega · Cycle = início → entrega

---

O que estudar agora

Quando	O que fazer
Hoje · 22:40 (revisão)	Reler as 7 regras 📌 dos erros acima. Criar no Anki: "RUP: dinâmica × estática × prática", "Scaling up × out", "Lead × cycle time", "V: qual teste para qual fase?", "Dono do Sprint Backlog"
Ter 27/10 · almoço	RLM (lógica proposicional), conforme o mapa de prioridades
Ter 27/10 · Bloco 1 (19h)	RF × RNF × regras de negócio (plano). Antes, 10 min de RUP: as 3 perspectivas e as 6 boas práticas, seção 3.6 do `mapa-engenharia-de-software`
Ter 27/10 · Bloco 2 (20h)	Questões FGV de requisitos + 5 questões FGV de Kanban (lead/cycle time, WIP)
Qua 28/10 · revisão	Revisão espaçada de 7 dias das regras de 21/10. Refazer Q1, Q2, Q11 e Q13 deste bloco sem olhar a correção
Dom 01/11 · simulado	No simulado, atenção a modelo em V e espiral. Contar quantas questões ficaram em branco (meta: zero)